

Semantic Kernel Python Vector Store Migration Guide

Overview

This guide covers the major vector store updates introduced in Semantic Kernel version 1.34, which represents a significant overhaul of the vector store implementation to align with the .NET SDK and provide a more unified, intuitive API. The changes consolidate everything under `semantic_kernel.data.vector` and improve the connector architecture.

Key Improvements Summary

- **Unified Field Model:** Single `VectorStoreField` class replaces multiple field types
- **Integrated Embeddings:** Direct embedding generation in vector field specifications
- **Simplified Search:** Easy creation of search functions directly on collections
- **Consolidated Structure:** Everything under `semantic_kernel.data.vector` and `semantic_kernel.connectors`
- **Enhanced Text Search:** Improved text search capabilities with streamlined connectors
- **Deprecation:** Old `memory_stores` are deprecated in favor of the new vector store architecture

1. Integrated Embeddings and Vector Store Models/Fields Updates

There are a number of changes to the way you define your vector store model, the biggest is that we now support integrated embeddings directly in the vector store field definitions. This means that when you specify a field to be a vector, the content of that field is automatically embedded using the specified embedding generator, such as OpenAI's text embedding model. This simplifies the process of creating and managing vector fields.

When you define that field, you need to make sure of three things, especially when using a Pydantic model:

1. **typing:** The field will likely have three types, `list[float]`, `str` or something else for the input to the embedding generator, and `None` for when the field is unset.

2. **default value:** The field must have a default value of `None` or something else, so that there is no error when getting records from `get` or `search` with `include_vectors=False` which is the default now.

There are two concerns here, the first is that when decorating a class with `vectorstoremodel`, the first type annotation of the field is used to fill the `type` parameter of the `VectorStoreField` class, so you need to make sure that the first type annotation is the right type for the vector store collection to be created with, often `list[float]`. By default, the `get` and `search` methods do not include `vectors` in the results, so the field needs a default value, and the typing needs to correspond to that, hence often `None` is allowed, and the default is set to `None`. When the field is created, the values that need to be embedded are in this field, often strings, so `str` also needs to be included. The reason for this change is to allow more flexibility in what is embedded and what is actually stored in data fields, this would be a common setup:

Python

```
from semantic_kernel.data.vector import VectorStoreField, vectorstoremodel
from typing import Annotated
from dataclasses import dataclass

@vectorstoremodel
@dataclass
class MyRecord:
    content: Annotated[str, VectorStoreField('data', is_indexed=True,
is_full_text_indexed=True)]
    title: Annotated[str, VectorStoreField('data', is_indexed=True,
is_full_text_indexed=True)]
    id: Annotated[str, VectorStoreField('key')]
    vector: Annotated[list[float] | str | None, VectorStoreField(
        'vector',
        dimensions=1536,
        distance_function="cosine",
        embedding_generator=OpenAITextEmbedding(ai_model_id="text-embedding-3-
small"),
    )] = None

    def __post_init__(self):
        if self.vector is None:
            self.vector = f"Title: {self.title}, Content: {self.content}"
```

Note the `post_init` method, this creates a value that get's embedded, which is more than a single field. The three types are also present.

Before: Separate Field Classes

Python

```
from semantic_kernel.data import (
    VectorStoreRecordKeyField,
    VectorStoreRecordDataField,
    VectorStoreRecordVectorField
)

# Old approach with separate field classes
fields = [
    VectorStoreRecordKeyField(name="id"),
    VectorStoreRecordDataField(name="text", is_filterable=True,
is_full_text_searchable=True),
    VectorStoreRecordVectorField(name="vector", dimensions=1536,
distance_function="cosine")
]
```

After: Unified VectorStoreField with Integrated Embeddings

Python

```
from semantic_kernel.data.vector import VectorStoreField
from semantic_kernel.connectors.ai.open_ai import OpenAITextEmbedding

# New unified approach with integrated embeddings
embedding_service = OpenAITextEmbedding(
    ai_model_id="text-embedding-3-small"
)

fields = [
    VectorStoreField(
        "key",
        name="id",
    ),
    VectorStoreField(
        "data",
        name="text",
        is_indexed=True, # Previously is_filterable
        is_full_text_indexed=True # Previously is_full_text_searchable
    ),
    VectorStoreField(
        "vector",
        name="vector",
        dimensions=1536,
        distance_function="cosine",
        embedding_generator=embedding_service # Integrated embedding generation
    )
]
```

Key Changes in Field Definition

1. **Single Field Class:** `VectorStoreField` replaces all previous field types

2. **Field Type Specification:** Use `field_type: Literal["key", "data", "vector"]` parameter, this can be a positional parameter, so `VectorStoreField("key")` is valid.

3. **Enhanced properties:**

- `storage_name` has been added, when set, that is used as the field name in the vector store, otherwise the `name` parameter is used.
- `dimensions` is now a required parameter for vector fields.
- `distance_function` and `index_kind` are both optional and will be set to `DistanceFunction.DEFAULT` and `IndexKind.DEFAULT` respectively if not specified and only for vector fields, each vector store implementation has logic that chooses a Default for that store.

4. **Property Renames:**

- `property_type` → `type_` as an attribute and `type` in constructors
- `is_filterable` → `is_indexed`
- `is_full_text_searchable` → `is_full_text_indexed`

5. **Integrated Embeddings:** Add `embedding_generator` directly to vector fields, alternatively you can set the `embedding_generator` on the vector store collection itself, which will be used for all vector fields in that store, this value takes precedence over the collection level embedding generator.

2. New Methods on Stores and Collections

Enhanced Store Interface

Python

```
from semantic_kernel.connectors.in_memory import InMemoryStore

# Before: Limited collection methods
collection = InMemoryStore.get_collection("my_collection", record_type=MyRecord)

# After: Slimmer collection interface with new methods
collection = InMemoryStore.get_collection(MyRecord)
# if the record type has the `vectorstoremodel` decorator it can contain both the
collection_name and the definition for the collection.

# New methods for collection management
await store.collection_exists("my_collection")
await store.ensure_collection_deleted("my_collection")
# both of these methods, create a simple model to streamline doing collection manage-
```

ment tasks.

they both call the underlying `VectorStoreCollection` methods, see below.

Enhanced Collection Interface

Python

```
from semantic_kernel.connectors.in_memory import InMemoryCollection

collection = InMemoryCollection(
    record_type=MyRecord,
    embedding_generator=OpenAITextEmbedding(ai_model_id="text-embedding-3-small") #
    Optional, if there is no embedding generator set on the record type
)
# If both the collection and the record type have an embedding generator set, the
# record type's embedding generator will be used for the collection. If neither is set,
# it is assumed the vector store itself can create embeddings, or that vectors are in-
# cluded in the records already, if that is not the case, it will likely raise.

# Enhanced collection operations
await collection.collection_exists()
await collection.ensure_collection_exists()
await collection.ensure_collection_deleted()

# CRUD methods
# Removed batch operations, all CRUD operations can now take both a single record or
# a list of records
records = [
    MyRecord(id="1", text="First record"),
    MyRecord(id="2", text="Second record")
]
ids = ["1", "2"]
# this method adds vectors automatically
await collection.upsert(records)

# You can do get with one or more ids, and it will return a list of records
await collection.get(ids) # Returns a list of records
# you can also do a get without ids, with top, skip and order_by parameters
await collection.get(top=10, skip=0, order_by='id')
# the order_by parameter can be a string or a dict, with the key being the field name
# and the value being True for ascending or False for descending order.
# At this time, not all vector stores support this method.

# Delete also allows for single or multiple ids
await collection.delete(ids)

query = "search term"
# New search methods, these use the built-in embedding generator to take the value
# and create a vector
results = await collection.search(query, top=10)
results = await collection.hybrid_search(query, top=10)

# You can also supply a vector directly
```

```
query_vector = [0.1, 0.2, 0.3] # Example vector
results = await collection.search(vector=query_vector, top=10)
results = await collection.hybrid_search(query, vector=query_vector, top=10)
```

3. Enhanced Filters for search

The new vector store implementation moves from string-based `FilterClause` objects to more powerful and type-safe lambda expressions or callable filters.

Before: FilterClause Objects

Python

```
from semantic_kernel.data.text_search import SearchFilter, EqualTo, AnyTagsEqualTo
from semantic_kernel.data.vector_search import VectorSearchFilter

# Creating filters using FilterClause objects
text_filter = SearchFilter()
text_filter.equal_to("category", "AI")
text_filter.equal_to("status", "active")

# Vector search filters
vector_filter = VectorSearchFilter()
vector_filter.equal_to("category", "AI")
vector_filter.any_tag_equal_to("tags", "important")

# Using in search
results = await collection.search(
    "query text",
    options=VectorSearchOptions(filter=vector_filter)
)
```

After: Lambda Expression Filters

Python

```
# When defining the collection with the generic type hints, most IDE's will be able
to infer the type of the record, so you can use the record type directly in the
lambda expressions.
collection = InMemoryCollection[str, MyRecord](MyRecord)

# Using lambda expressions for more powerful and type-safe filtering
# The code snippets below work on a data model with more fields than defined earlier.

# Direct lambda expressions
results = await collection.search(
    "query text",
    filter=lambda record: record.category == "AI" and record.status == "active"
```

```
)

# Complex filtering with multiple conditions
results = await collection.search(
    "query text",
    filter=lambda record: (
        record.category == "AI" and
        record.score > 0.8 and
        "important" in record.tags
    )
)

# Combining conditions with boolean operators
results = await collection.search(
    "query text",
    filter=lambda record: (
        record.category == "AI" or record.category == "ML"
    ) and record.published_date >= datetime(2024, 1, 1)
)

# Range filtering (now possible with lambda expressions)
results = await collection.search(
    "query text",
    filter=lambda record: 0.5 <= record.confidence_score <= 0.9
)
```

Migration Tips for Filters

1. **Simple equality:** `filter.equal_to("field", "value")` becomes `lambda r: r.field == "value"`
2. **Multiple conditions:** Chain with `and/or` operators instead of multiple filter calls
3. **Tag/array containment:** `filter.any_tag_equal_to("tags", "value")` becomes `lambda r: "value" in r.tags`
4. **Enhanced capabilities:** Support for range queries, complex boolean logic, and custom predicates

4. Improved Ease of Creating Search Functions

Before: Search Function Creation with `VectorStoreTextSearch`

Python

```
from semantic_kernel.connectors.in_memory import InMemoryCollection
from semantic_kernel.data import VectorStoreTextSearch

collection = InMemoryCollection(collection_name='collection', record_type=MyRecord)
search = VectorStoreTextSearch.from_vectorized_search(vectorized_search=collection,
    embedding_generator=OpenAITextEmbedding(ai_model_id="text-embedding-3-small"))
```

```
search_function = search.create_search(  
    function_name='search',  
    ...  
)
```

After: Direct Search Function Creation

Python

```
collection = InMemoryCollection(MyRecord)  
# Create search function directly on collection  
search_function = collection.create_search_function(  
    function_name="search",  
    search_type="vector", # or "keyword_hybrid"  
    top=10,  
    vector_property_name="vector", # Name of the vector field  
)  
  
# Add to kernel directly  
kernel.add_function(plugin_name="memory", function=search_function)
```

5. Connector Renames and Import Changes

Import Path Consolidation

Python

```
# Before: Scattered imports  
from semantic_kernel.connectors.memory.azure_cognitive_search import  
AzureCognitiveSearchMemoryStore  
from semantic_kernel.connectors.memory.chroma import ChromaMemoryStore  
from semantic_kernel.connectors.memory.pinecone import PineconeMemoryStore  
from semantic_kernel.connectors.memory.qdrant import QdrantMemoryStore  
  
# After: Consolidated under connectors  
from semantic_kernel.connectors.azure_ai_search import AzureAISearchStore  
from semantic_kernel.connectors.chroma import ChromaVectorStore  
from semantic_kernel.connectors.pinecone import PineconeVectorStore  
from semantic_kernel.connectors.qdrant import QdrantVectorStore  
  
# Alternative after: Consolidated with lazy loading:  
from semantic_kernel.connectors.memory import (  
    AzureAISearchStore,  
    ChromaVectorStore,  
    PineconeVectorStore,  
    QdrantVectorStore,  
    WeaviateVectorStore,  
    RedisVectorStore
```



```
)
```

Connector Class Renames

[Expand table](#)

Old Name	New Name
AzureCosmosDBforMongoDB*	CosmosMongo*
AzureCosmosDBForNoSQL*	CosmosNoSql*

6. Text Search Improvements and Removed Bing Connector

Bing Connector Removed and Enhanced Text Search Interface

The Bing text search connector has been removed. Migrate to alternative search providers:

Python

```
# Before: Bing Connector (REMOVED)
from semantic_kernel.connectors.search.bing import BingConnector

bing_search = BingConnector(api_key="your-bing-key")

# After: Use Brave Search or other providers
from semantic_kernel.connectors.brave import BraveSearch
# or
from semantic_kernel.connectors.search import BraveSearch

brave_search = BraveSearch()

# Create text search function
text_search_function = brave_search.create_search_function(
    function_name="web_search",
    query_parameter_name="query",
    description="Search the web for information"
)

kernel.add_function(plugin_name="search", function=text_search_function)
```

Improved Search Methods

Before: Three separate search methods with different return types

Python

```
from semantic_kernel.connectors.brave import BraveSearch
brave_search = BraveSearch()
# Before: Separate search methods
search_results: KernelSearchResult[str] = await brave_search.search(
    query="semantic kernel python",
    top=5,
)

search_results: KernelSearchResult[TextSearchResult] = await
brave_search.get_text_search_results(
    query="semantic kernel python",
    top=5,
)

search_results: KernelSearchResult[BraveWebPage] = await
brave_search.get_search_results(
    query="semantic kernel python",
    top=5,
)
```

After: Unified search method with output type parameter

Python

```
from semantic_kernel.data.text_search import SearchOptions
# Enhanced search results with metadata
search_results: KernelSearchResult[str] = await brave_search.search(
    query="semantic kernel python",
    output_type=str, # can also be TextSearchResult or anything else for search engine specific results, default is `str`
    top=5,
    filter=lambda result: result.country == "NL", # Example filter
)

async for result in search_results.results:
    assert isinstance(result, str) # or TextSearchResult if using that type
    print(f"Result: {result}")
    print(f"Metadata: {search_results.metadata}")
```

7. Deprecation of Old Memory Stores

All the old memory stores, based on `MemoryStoreBase` have been moved into `semantic_kernel.connectors.memory_stores` and are now marked as deprecated. Most of them

have a equivalent new implementation based on `VectorStore` and `VectorStoreCollection`, which can be found in `semantic_kernel.connectors.memory`.

These connectors will be removed completely:

- AstraDB
- Milvus
- Usearch

If you need any of these still, either, make sure to take over the code from the deprecated module and the `semantic_kernel.memory` folder, or [implement your own vector store collection](#) based on the new `VectorStoreCollection` class.

If there is a large demand based on github feedback, we will consider bringing them back, but for now, they are not maintained and will be removed in the future.

Migration from SemanticTextMemory

Python

```
# Before: SemanticTextMemory (DEPRECATED)
from semantic_kernel.memory import SemanticTextMemory
from semantic_kernel.connectors.ai.open_ai import
OpenAITextEmbeddingGenerationService

embedding_service = OpenAITextEmbeddingGenerationService(ai_model_id="text-embedding-
3-small")
memory = SemanticTextMemory(storage=vector_store,
embeddings_generator=embedding_service)

# Store memory
await memory.save_information(collection="docs", text="Important information",
id="doc1")

# Search memory
results = await memory.search(collection="docs", query="important", limit=5)
```

Python

```
# After: Direct Vector Store Usage
from semantic_kernel.data.vector import VectorStoreField, vectorstoremodel
from semantic_kernel.connectors.in_memory import InMemoryCollection

# Define data model
@vectorstoremodel
@dataclass
class MemoryRecord:
    id: Annotated[str, VectorStoreField('key')]
```

```

    text: Annotated[str, VectorStoreField('data', is_full_text_indexed=True)]
    embedding: Annotated[list[float] | str | None, VectorStoreField('vector',
dimensions=1536, distance_function="cosine",
embedding_generator=OpenAITextEmbedding(ai_model_id="text-embedding-3-small"))] =
None

# Create vector store with integrated embeddings
collection = InMemoryCollection(
    record_type=MemoryRecord,
    embedding_generator=OpenAITextEmbedding(ai_model_id="text-embedding-3-small") #
Optional, if not set on the record type
)

# Store with automatic embedding generation
record = MemoryRecord(id="doc1", text="Important information", embedding='Important
information')
await collection.upsert(record)

# Search with built-in function
search_function = collection.create_search_function(
    function_name="search_docs",
    search_type="vector"
)

```

Memory Plugin Migration

When you want to have a plugin that can also save information, then you can easily create that like this:

Python

```

# Before: TextMemoryPlugin (DEPRECATED)
from semantic_kernel.core_plugins import TextMemoryPlugin

memory_plugin = TextMemoryPlugin(memory)
kernel.add_plugin(memory_plugin, "memory")

```

Python

```

# After: Custom plugin using vector store search functions
from semantic_kernel.functions import kernel_function

class VectorMemoryPlugin:
    def __init__(self, collection: VectorStoreCollection):
        self.collection = collection

    @kernel_function(name="save")
    async def save_memory(self, text: str, key: str) -> str:
        record = MemoryRecord(id=key, text=text, embedding=text)
        await self.collection.upsert(record)

```

```

        return f"Saved to {self.collection.collection_name}"

    @kernel_function(name="search")
    async def search_memory(self, query: str, limit: int = 5) -> str:
        results = await self.collection.search(
            query, top=limit, vector_property_name="embedding"
        )
        return "\n".join([r.record.text async for r in results.results])

# Register the new plugin
memory_plugin = VectorMemoryPlugin(collection)
kernel.add_plugin(memory_plugin, "memory")

```

Migration Checklist for Vector Search

Step 1: Update Imports

- [] Replace memory store imports with vector store equivalents
- [] Update field imports to use `VectorStoreField`
- [] Remove Bing connector imports

Step 2: Update Field Definitions

- [] Convert to unified `VectorStoreField` class
- [] Update property names (`is_filterable` → `is_indexed`)
- [] Add integrated embedding generators to vector fields

Step 3: Update Collection Usage

- [] Replace memory operations with vector store methods
- [] Use new batch operations where applicable
- [] Implement new search function creation

Step 4: Update Search Implementation

- [] Replace manual search functions with `create_search_function`
- [] Update text search to use new providers
- [] Implement hybrid search where beneficial
- [] Migrate from `FilterClause` to `lambda` expressions for filtering

Step 5: Remove Deprecated Code

- [] Remove `SemanticTextMemory` usage

- `[]` Remove `TextMemoryPlugin` dependencies

Performance and Feature Benefits

Performance Improvements

- **Batch Operations:** New batch upsert/delete methods improve throughput
- **Integrated Embeddings:** Eliminates separate embedding generation steps
- **Optimized Search:** Built-in search functions are optimized for each store type

Feature Enhancements

- **Hybrid Search:** Combines vector and text search for better results
- **Advanced Filtering:** Enhanced filter expressions and indexing

Developer Experience

- **Simplified API:** Fewer classes and methods to learn
- **Consistent Interface:** Unified approach across all vector stores
- **Better Documentation:** Clear examples and migration paths
- **Future-Proof:** Aligned with .NET SDK for consistent cross-platform development

Conclusion

The vector store updates discussed above represent a significant improvement in the Semantic Kernel Python SDK. The new unified architecture provides better performance, enhanced features, and a more intuitive developer experience. While migration requires updating imports and refactoring existing code, the benefits in maintainability and functionality make this upgrade highly recommended.

For additional help with migration, refer to the updated samples in the `samples/concepts/memory/` directory and the comprehensive API documentation.

Last updated on 06/25/2025